EEEE4138 Aerial Robotics Practical 1

Vertical Dynamics and Simulink

This guided practical develops a vertical-motion model in MATLAB and Simulink. Construct the model from Newton's second law, predict its response and verify the simulation against analytical results. MATLAB and Simulink are required; no additional toolbox is needed.

Learning objectives

- Implement a vertical-motion model using Newton's second law and two integrators.
- Compare simulated responses with analytical predictions for four test conditions.
- Explain the effects of initial velocity, increased payload and thrust limits.

1 Model assumptions and governing equations

Assume a level aircraft with constant mass, no rotation and no aerodynamic drag. The coordinate z is positive downwards; w is downward velocity. Total thrust T is an upward force magnitude in newtons.

Weight contributes +mg and thrust contributes −T. Newton's law therefore gives acceleration a = g − T/m. Integrate acceleration to obtain velocity, then integrate velocity to obtain position.

A negative z means above launch level. The model has no ground-contact calculation: passing below z = 0 does not stop the simulation.

2 Working folder setup

Create a folder named Lab1_Sxx, where Sxx is the assigned student ID. Set this folder as MATLAB's Current Folder. Save the parameter script and Simulink model in this folder, and create a results subfolder for simulation data, plots and comparison tables.

3 Parameter definition

Create a MATLAB script named lab1_parameters.m and enter the code below. Replace m0 with the assigned unloaded mass. The values shown correspond to student S01.

```
g = 9.81;
m0 = 1.31;              % Replace with your assigned mass, kg
m_nom = 1.50*m0;        % Nominal mass including payload
m = m_nom;              % Actual mass used by the plant
T_hover = m_nom*g;      % Original hover thrust, N
T_max = 2*T_hover;      % Fixed hardware thrust limit, N
T_cmd = T_hover;        % First test command, N
w0 = 0;                 % Initial downward velocity, m/s
z0 = 0;                 % Initial downward position, m
```

Run the script and confirm that the variables appear in the MATLAB Workspace. For S01, m = 1.965 kg and T_hover = 19.27665 N. Retain full numerical precision in the script.

4 Simulink model implementation

Open Simulink and choose Blank Model. Save it as Lab1_Vertical.slx in your working folder. Double-click empty canvas space, type the block name and select the standard Simulink block. Alternatively use the Library Browser.

Table 1 Simulink blocks and parameter settings

Block	Number	Parameter to enter
Constant	2	First value: T_cmd. Second value: g.
Gain	1	Gain: 1/m
Sum	1	List of signs: +− (type the characters + and -).
Integrator	2	Use Continuous Integrator. First initial condition: w0. Second: z0.
Scope	3	One each for acceleration, velocity and position.

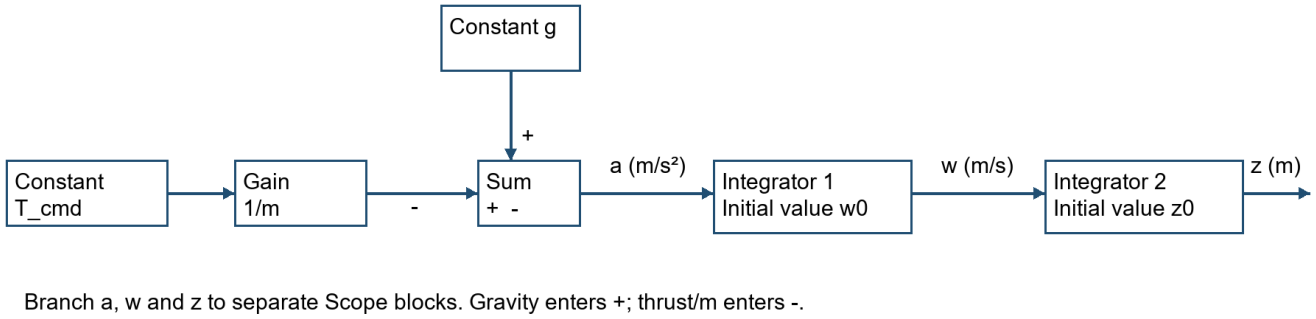


Figure 1 Vertical motion model and signal connections

Connect output ports to input ports by dragging between them. Follow the signs printed beside the Sum input ports: g must enter + and the Gain output must enter −. The diagram shows signal flow; block port locations may differ.

Connect Sum → first Integrator → second Integrator. Rename the integrators Velocity and Position. Double-click each signal line to name the outputs a, w and z.

Branch a to one Scope, w to another and z to the third. To branch, hold Ctrl and drag from an existing signal line to the Scope input. Rename the Scopes Acceleration, Velocity and Position.

Verification checkpoint: identify the input and output of each block. Confirm that the model calculates g − T_cmd/m and integrates this acceleration to obtain velocity and position, respectively.

5 Initial simulation and verification

Set Stop Time to 2 seconds. Open Model Settings → Solver. Select Variable-step and ode45. Set Maximum step size to 0.01 s, Relative tolerance to 1e-6 and Absolute tolerance to 1e-8. These settings apply to this smooth vertical model.

Run lab1_parameters.m first, then click Run in Simulink. Double-click each Scope. Use its Autoscale control if the trace is not visible. All three outputs should remain zero for the initial hover test.

Initial conditions and analytical solution

Set the initial velocity and position using w0 and z0 in the respective Integrator blocks. For constant thrust and mass, calculate the analytical response using:

a = g - T/m
w(t) = w0 + a*t
z(t) = z0 + w0*t + 0.5*a*t^2

6 Model validation tests

For each test in Table 2, predict a, w(2) and z(2). Run the parameter script to restore nominal conditions, then enter the specified commands in the MATLAB Command Window and run the simulation. Do not rerun the parameter script after entering the test commands, as this would restore the initial hover settings.

Table 2 Validation test conditions and reference results

Test	Commands after the parameter script	Expected a; w(2); z(2)
1 Hover	T_cmd = T_hover; w0 = 0;	
2 Free fall	T_cmd = 0; w0 = 0;	
3 Extra thrust	T_cmd = 1.10*T_hover; w0 = 0;	
4 Existing descent	T_cmd = T_hover; w0 = 0.40;	

Use z0 = 0 and the nominal loaded mass for all four tests. For nonzero constant acceleration, verify that velocity varies linearly and position varies quadratically with time. For zero acceleration and nonzero w0, verify that position varies linearly with time.

Discussion: explain why the UAV continues to descend in Test 4 despite zero net force. Distinguish between zero acceleration and zero velocity.

7 Data recording and comparison

For a numerical comparison, add three To Workspace blocks and branch a, w and z into them. Set Variable name to a_log, w_log and z_log respectively; choose Timeseries as the Save format. Keep the Scopes connected.

In Model Settings → Data Import/Export, enable Single simulation output and set its variable name to out. Run one test. In MATLAB, read the final values using:

```
out.a_log.Data(end)
out.w_log.Data(end)
out.z_log.Data(end)
```

Save the results of each test before running the next simulation. The following example saves the data and position plot for Test 2:

```
save('results/test2.mat','out','m','T_cmd','w0','z0');
figure;
plot(out.z_log.Time,out.z_log.Data);
xlabel('Time (s)'); ylabel('Downward position z (m)');
grid on;
saveas(gcf,'results/test2_position.png');
```

Use a distinct filename for each test. Produce labelled acceleration, velocity and position plots, and complete Table 3. Confirm that the results folder exists before saving files.

Table 3 Predicted and simulated results

Test	Predicted w(2), z(2)	Simulated w(2), z(2)	Difference and explanation
Hover			
Free fall			
Extra thrust			
Existing descent			

Troubleshooting checks

Table 4 Common simulation issues and checks

Symptom	Check
Undefined variable	Run the parameter script; check spelling and Current Folder.
More thrust accelerates downwards	Gravity must enter +; T/m must enter −.
Hover drifts	Check exact thrust, actual mass and w0.
Position resembles velocity	Check both integrators and their order.
Test settings seem ignored	Do not run the reset script after test overrides.
No out logging result	Check Single simulation output, variable names and Timeseries format.

8 Payload and actuator limits

Complete the four validation tests before proceeding. Run lab1_parameters to restore nominal conditions, then enter the following commands. Retain the original nominal hover thrust and maximum thrust limit.

```
m = 1.80*m0;
T_cmd = T_hover;
w0 = 0; z0 = 0;
```

Calculate the predicted acceleration for the increased mass, then simulate the response. Explain the direction of motion and compare the simulation with a = g − T_hover/m.

Set T_cmd = m*g and simulate from rest. Repeat with w0 = 0.40 while retaining this thrust. Explain why force balance does not stop an existing descent, and state whether thrust must increase or decrease to reduce the downward speed.

Actuator saturation

Insert a Saturation block between the thrust Constant and Gain. Set Lower limit to 0 and Upper limit to T_max. Label the signal before it T_command and after it T_actual. Use T_actual, not the unclipped command, in analytical predictions.

Reset to nominal conditions by running lab1_parameters. Then test T_cmd = 1.20*T_max. Check that the Saturation output is T_max. Repeat with T_cmd = -1; the applied thrust should be zero. Do not increase T_max when adding payload.

9 Vertical braking manoeuvre

The UAV is initially descending at 0.80 m/s. Use the nominal loaded mass, take the initial position as z = 0, and retain the downward-positive convention. Neglect aerodynamic drag and motor response delay.

(a) Determine the constant vertical acceleration required to bring the UAV to rest in 0.50 s.

(b) Calculate the required thrust and verify that it lies within the available thrust limits.

(c) Calculate the downward distance travelled before the UAV stops.

(d) Implement the manoeuvre in Simulink. Apply the calculated braking thrust for the first 0.50 s, then switch to the thrust required for stationary hover. Simulate for 2.00 s and plot thrust, acceleration, velocity and position.

Hint: a Step block can specify the initial thrust, final thrust and switching time.

(e) Compare the simulated stopping time and displacement with the analytical predictions. Explain what would happen if the braking thrust remained applied after 0.50 s.

Extension: repeat the simulation with the increased payload from Section 8, without changing either thrust command. Explain any differences.

Retain the calculations and labelled plots as evidence.

10 Model organisation

After validation, group the plant into a subsystem. Select the Gain, gravity Constant, Sum and both Integrators; right-click and choose Create Subsystem from Selection. Keep the thrust command, Saturation and Scopes outside. Rename the subsystem VerticalPlant. Check the automatically created ports and label its input T and outputs a, w and z. Rerun hover to check that behaviour is unchanged.